

## **CS 425 : DISTRIBUTED SYSTEMS - MP 4 REPORT**

**(GROUP-37)** Akash Elumalai (akashe2) and Nikunj Agarwal (nikunja2)

**a) Design:** We have implemented the Rainstorm framework in Golang. It has leader and worker nodes that co-ordinate together via RPCs to support filters, transformations, aggregations on the incoming stream. It also supports exactly-once semantics to ensure there are no duplicates under task failure and auto-scaling to load balance tasks based on the stream rate.

**Task Workflow:** As soon as a Job is submitted, the leader constructs the topology with details such as list of stages, list of tasks, each tasks' node assignments, its input and output targets (tasks) and broadcasts it to the workers. The workers have a **node-level manager**, which initializes the tasks on receiving the topology. During initialization, it fetches the WAL logs, reconstructs the Received and Sent Tuples maps, runs task executable as a separate process, updates the Stdin and Stdout pipes. Once all the nodes have initialized the tasks, Leader starts streaming by reading the input dataset and emitting the <filename:linenumber, line> tuples to the stage 1 tasks. On receipt of a tuple, the NM redirects the tuple to the appropriate task's stdin pipe, reads output from its stdout pipe and forwards it to the next stage. Each output tuple is hashed by its key % len(tasks in next stage) for forwarding to the next task.

**Exactly-once:** For exactly once, each task is paired with an Exactly-Once Manager that tracks both received and sent tuples. For every incoming tuple, the manager looks at the received map and if the tuple ID is already present, it is treated as a duplicate and immediately ACKed back to the sender without processing. For new inputs, the manager logs a REC record, processes the tuple through operation and then records each emitted output as a SENT record in sentTuples and later an ACK record when the downstream ACK arrives. All of these records are buffered in memory and periodically flushed as JSON lines into a per-task WAL in HyDFS, so the complete state of in-flight and processed tuples is durably replicated. Note that we only send back an ACK to the previous stage once we have processed and logged the tuple in HyDFS. If a node or task fails, the new instance reconstructs its Exactly-Once Manager by replaying the WAL, restoring both dedupe state and any un-ACKed sends. A background retry loop re-sends un-ACKed SENT tuples, optionally rerouting them using the current autoscaled topology, and when tasks are scaled down, their WALs are merged into surviving tasks, allowing in-flight work to be continued without loss or duplication.

**Autoscaling:** The NM is responsible for collecting metrics and statuses for all tasks running on its node and periodically reporting them to the leader. The leader aggregates these per-task metrics into per-stage throughput, then computes the average tuples per second per task for each stage. Using configured lower and upper thresholds (LW, HW), the leader decides whether a stage is underutilized or overloaded. If the average is below LW, it scales down by killing one task from that stage. If the average is above HW, it scales up by adding a new task instance on some node and updating the topology and assignments. This feedback loop runs continuously, allowing the system to dynamically adjust the number of tasks per stage to match the current workload. To preserve atleast-once semantics during scale down, the leader triggers a WAL merge on the node that hosts a surviving task in the same stage. The NM fetches the killed task's WAL from HyDFS and replays REC, SENT, and ACK records into its in-memory maps.

This allows the surviving task to continue retrying any un-ACKed sends from the killed task, ensuring that in-flight tuples are not lost and are delivered at least once, even as the autoscaler removes capacity.

**b) Past MP Use:** We have extended MP2's membership protocol which keeps track of the newly joined nodes and the available nodes. The membership list becomes handy while scheduling tasks. We use MP3's HyDFS to Create and Append WAL logs. We also use its functions to append the final output to the same file. MP1 distributed grep was useful during the report compilation where we had to grep some logs which included bandwidth and time taken.

**c) LLM use:** We mainly used LLMs to brainstorm design ideas. It was quite useful to get a better understanding of the project and to have a general conversation about the design decisions. It became very handy in deciding whether each task should be a thread or process, how exactly once handled in production systems, how we can send data to the process(using stdin in our case). We found it most effective for clarifying concepts and boilerplate code generation, but not for actual end-to-end implementation. Refer to the "LLM usage" folder for chat sessions.

#### **d) Measurements:**

##### **Datasets used:**

**Amazon sale:** <https://www.kaggle.com/datasets/rohiteng/amazon-sales-dataset>(100000 lines)

**Netflix shows :** <https://www.kaggle.com/datasets/shivamb/netflix-shows>(~9000 lines)

We have used the above 2 datasets. The Amazon sales dataset contains e-commerce transaction records including customer, product, and payment details. The NetflixShows dataset contains metadata about movies and TV shows available on Netflix.

The combination of 2 Applications used are as follows:

- **Application 1: filter + transform(first 3 columns)**

In this application, we filter the lines using case sensitive matching and then output the first 3 columns of csv line

- For dataset 1: filter for line containing "Debit" and output first 3 columns
- For dataset 2: filter for line containing "Movie" and output first 3 columns

- **Application 2: filter + lowercase**

In this application, we filter the lines, select a particular column and then convert that to lowercase

- For dataset 1:filter for lines with "States" and convert customer name to lowercase
- For dataset 2:filter for lines with "States" and convert director name to lowercase

**Setup:** For both applications, we used a batch size of 500 and set the number of partitions (tasks per stage) to 3. In the Spark scripts, we explicitly applied the repartition() function after each stage or operation to enforce a shuffle boundary and ensure a fair comparison with Rainstorm. Tuples were processed in batches, and we introduced a small sleep interval between batches to simulate streaming arrival rates. We measured throughput by dividing the

total number of processed tuples by the total time taken, capturing the effective tuples processed per second under this controlled streaming setup.

**Analysis:** In all the four graphs, observe that Spark consistently achieves slightly higher throughput than Rainstorm under the given setup. This is likely due to additional overheads in Rainstorm, such as inter-node RPC communication, tuple serialization/deserialization, exactly-once WAL logging, and final writes to HyDFS. In Rainstorm, each tuple is redirected to a task via stdin/stdout whereas Spark operates on in-memory RDDs which reduces per-tuple overhead. We also see that for datasets with fewer output tuples, the performance gap narrows, possibly because HyDFS writes dominate at the end and using local file output could further reduce this difference. Additionally, we also think that while Spark benefits from extensive internal optimizations, we ensured a fair comparison by batching tuples, introducing intervals between batches and repartitioning between stages. If we remove the sleep between batches, we see that it nearly doubles Spark's processing speed.

